

AMASE: A Harness-Ablation Framework for Measuring Cost–Quality Trade-offs in Autonomous Software Engineering Agents

Department of Computer Science and Engineering

(Supervisor: Mr. <Supervisor Name>, Assistant Professor, CSE)



Name of Member	Roll Number	Role
Aryan Raj Singh	<Roll No>	Backend, Agent Harness, Telemetry
<Team Member Name>	<Roll No>	Sandbox & Infrastructure
<Team Member Name>	<Roll No>	Evaluation & Benchmark Authoring
<Team Member Name>	<Roll No>	Documentation & Dashboard

Index

1. Introduction
2. Problem Definition
3. Objectives
4. Literature Survey
5. Research Gap
6. Proposed Methodology – System Architecture
7. Proposed Methodology – Ablation Arms
8. Feasibility Study, Need and Significance
9. Facilities Required
10. Expected Outcomes and References



Introduction

What is an autonomous software engineering agent?

An LLM operating a closed perception–action loop over a real development environment: it reads files, searches a repository, edits source, runs shell commands and tests, observes stdout/stderr, and revises its own work until the goal is met or a budget is exhausted.

What is the agent harness?

The software around the model — tool interface, prompting, control flow, context management, loop detection, model-selection policy and instrumentation. It involves no retraining, yet it decides what the system can actually do.

Why this matters

- Harness design is worth an estimated 11–15 points of task resolution with the model held fixed.
- One published result: 52.8 → 66.5 on Terminal-Bench 2.0 — harness changed, model unchanged.
- That is comparable to, and often larger than, an entire model-generation upgrade.

Problem Definition

The gap

Practitioners choose among many harness mechanisms — context injection, pre-completion verification, loop detection, compaction, sub-agent delegation, retrieval architecture, model routing, reasoning-effort allocation — with no controlled evidence attributing an effect size or a cost to any individual mechanism.

Three consequences

- Effort is spent on fashionable mechanisms rather than effective ones.
- Systems ship with unquantified unit economics: +2 points at 3× tokens is invisible.
- The benchmarks used to settle such questions are compromised.

Benchmark validity has collapsed

- OpenAI retired SWE-bench Verified in 2026: an audit of 138 problems found 59.4% materially defective, with frontier models reproducing reference patches verbatim.

Objectives

- Build a deliberately minimal coding-agent harness in which every mechanism is an independently switchable experimental arm rather than an architectural assumption.
- Instrument every model call and tool invocation for tokens, monetary cost, latency and outcome, conforming to the OpenTelemetry GenAI semantic conventions.
- Measure the marginal contribution of eight harness mechanisms to task resolution rate, token consumption, cost per task and p95 latency.
- Execute each configuration a minimum of three times and report variance, since single-run agent benchmark figures are dominated by stochastic noise.
- Author a decontaminated held-out task set so that findings survive scrutiny of benchmark validity.
- Gate every pull request on an automated regression check so accumulated results are protected.
- Publish the resulting attribution table, including negative results, as a reproducible artefact.

Literature Survey

Sr.	Work	Year	Contribution	Limitation
1	ReAct (Yao et al.)	2022	Interleaves reasoning with actions; the control paradigm coding agents use.	Defines the loop, not what makes it reliable.
2	Reflexion (Shinn et al.)	2023	Verbal self-reflection over failures without weight updates.	No component attribution or cost accounting.
3	SWE-bench (Jimenez et al.)	2023	First large-scale real GitHub issue benchmark.	Saturated and contaminated; retired 2026.
4	SWE-agent (Yang et al.)	2024	Purpose-built Agent-Computer Interface raises resolution sharply.	Shows interface matters; does not decompose it.
5	OpenHands (Wang et al.)	2024	Open generalist software-development agent platform.	Optimises a whole product; mechanisms entangled.
6	Lost in the Middle (Liu et al.)	2023	Retrieval degrades for mid-context information.	Diagnoses; does not evaluate remedies in a loop.
7	RouteLLM (Ong et al.)	2024	Learned routing: ~95% quality at ~26% of cost.	Chat benchmarks, not long-horizon agentic tasks.
8	SWE-rebench (Badertdinov et al.)	2025	Automated decontaminated task collection pipeline.	Clean tasks, but whole-system evaluation.
9	SWE-Bench Pro (Scale AI)	2025	1,865 long-horizon tasks, ~107 LOC over ~4.1 files.	Raises difficulty; attribution untouched.
10	Terminal-Bench (Merrill et al.)	2026	89 sandboxed CLI tasks, end-state verification, pluggable agents.	Supplies the substrate; supplies no ablations.

Research Gap

What the literature establishes

- The harness is the dominant controllable factor in coding-agent performance.

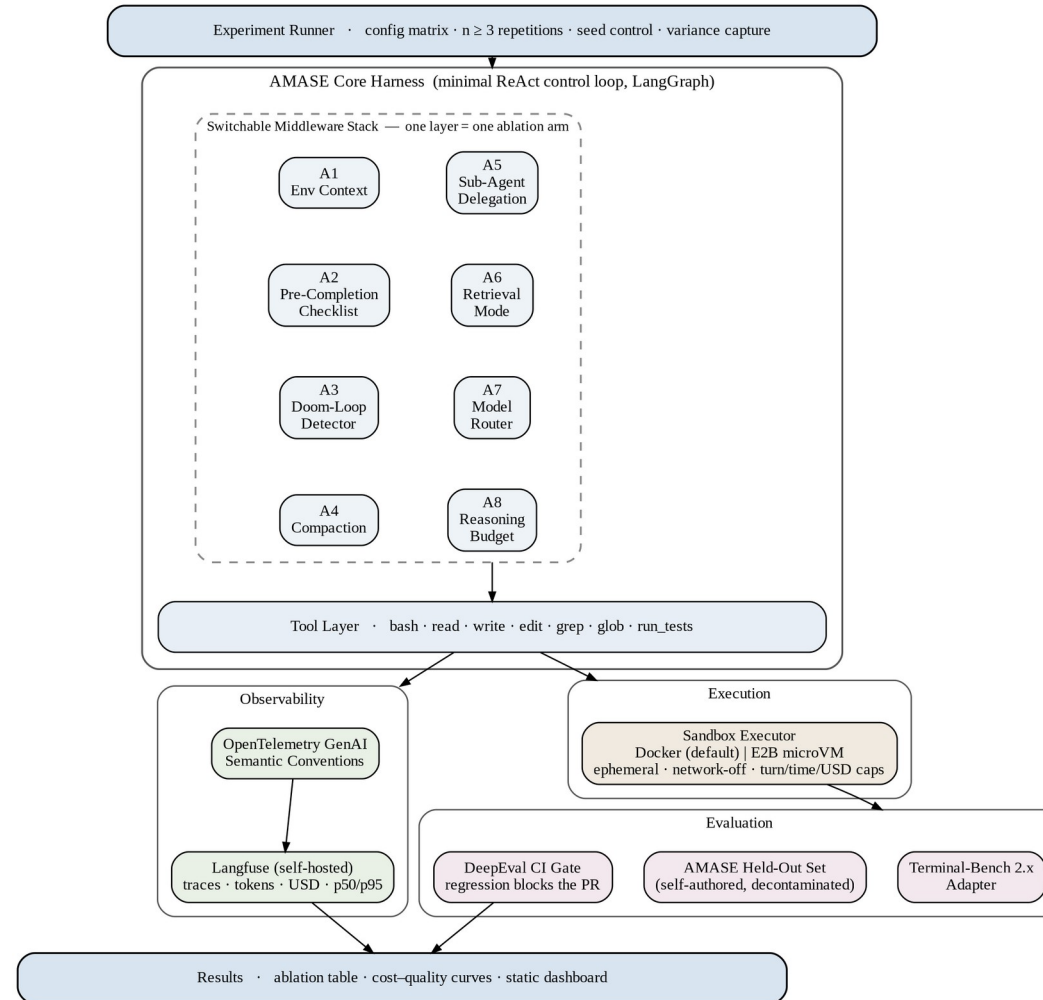
What it does not establish

- No work isolates individual harness mechanisms under controlled conditions — published results compare complete systems, so a practitioner cannot tell which of a dozen simultaneous design choices produced an observed improvement.
- Cost is almost universally omitted. Benchmarks report resolution rate, not the tokens or currency spent to achieve it, although that ratio decides whether a technique is deployable.
- The benchmarks conventionally used for such measurement are demonstrably invalid.

The gap AMASE occupies

A reproducible, cost-annotated, contamination-resistant framework that attributes agent performance to its individual harness components — addressing all four conditions simultaneously.

Proposed Methodology – System Architecture



Proposed Methodology – Harness Ablation Arms

Arm	Mechanism	Function	Hypothesis
A1	Environment context	Directory map, tool discovery and budget stated at startup.	Small gain, low token cost
A2	Pre-completion checklist	Blocks completion until output is verified and tests run.	Largest single effect
A3	Doom-loop detection	Per-file edit counters trigger a forced re-plan.	Fewer catastrophic runs; better p95
A4	Compaction strategy	Truncation vs summarisation vs external notes file.	Notes win on long-horizon tasks
A5	Sub-agent delegation	Review in a clean context, returning a condensed summary.	Positive only past a length threshold
A6	Retrieval mode	Agentic search vs BM25 + dense + cross-encoder rerank.	Agentic wins on exact symbols
A7	Model routing	Frontier for plan/debug, local SLM for mechanical edits.	Large cost cut, small quality loss
A8	Reasoning budget	Uniform effort vs high-plan / mid-build / high-verify.	Better resolution per rupee

Feasibility Study, Need and Significance

Technical

- Every component is mature and open source — LangGraph, Docker, Langfuse, DeepEval, Ollama, Terminal-Bench. No model training, no specialised hardware, no proprietary dataset.

Economic

- Full matrix runs on a cheap or local model; frontier models only for final confirmation. Hard per-rollout caps on turns, time and spend. Spend tracked in the same telemetry pipeline.

Operational and Schedule

- Operated by the developers, so no adoption risk. Five phases with explicit exit criteria; the measurement infrastructure is completed before any experimental arm is attempted.

Need and Significance

- Agents are deployed in production on anecdotal design evidence, while the usual means of settling such questions has been withdrawn by its own authors.
- Contribution: an attribution of effect and cost per mechanism, a reusable experimental apparatus, and a decontaminated task set. Negative results carry equal weight.

Facilities Required

Hardware

- Intel Core i5 (8th gen) / AMD Ryzen 5 or better; 16 GB RAM (32 GB recommended); 256 GB SSD with 80 GB free; NVIDIA GPU with 8 GB VRAM (optional, for the local-model routing arm).

Software

- Python 3.11 with uv; LangGraph for the stateful control loop; Docker Engine and Compose for sandboxes; E2B SDK behind the same interface for microVM isolation.
- OpenTelemetry SDK with a self-hosted Langfuse backend; DeepEval and pytest for the CI gate; Terminal-Bench 2.x as the task harness.
- Qdrant, rank_bm25, sentence-transformers and tree-sitter for the hybrid retrieval arm; Ollama and vLLM for local model serving; pandas and matplotlib for analysis.

Data

- Terminal-Bench 2.x public dataset, plus 25–40 self-authored held-out tasks in three difficulty tiers, drawn from repositories with no plausible training-corpus presence.

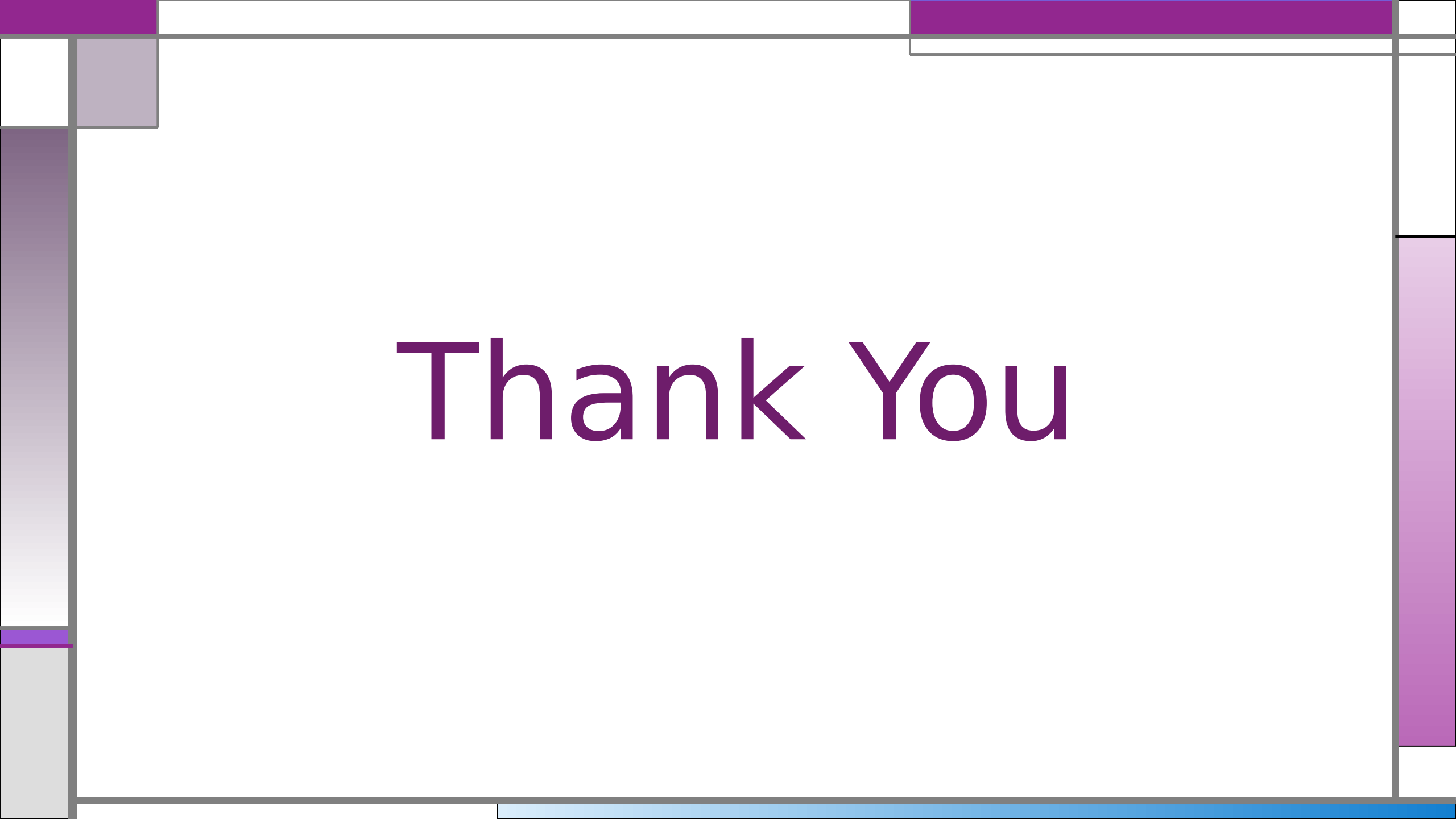
Expected Outcomes and References

Expected outcomes

- An attribution table: for each of eight mechanisms, its marginal effect on resolution rate with token and monetary cost attached, reported with variance over $n \geq 3$ runs.
- Cost–quality curves for the routing and retrieval arms, with the operating knee identified.
- A reproducible apparatus and a decontaminated task set, released with the write-up.

References

- [1] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models”, arXiv:2210.03629, 2022.
- [2] C. E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?”, arXiv:2310.06770, 2023.
- [3] J. Yang et al., “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering”, NeurIPS, 2024.
- [4] I. Ong et al., “RouteLLM: Learning to Route LLMs with Preference Data”, arXiv:2406.18665, 2024.
- [5] M. A. Merrill et al., “Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in CLIs”, arXiv:2601.11868, 2026.
- [6] OpenAI, “Why SWE-bench Verified No Longer Measures Frontier Coding Capabilities”, 2026.



Thank You

Feedback from the Panel

- The panel wants the team to :